

□ Topic of the session: Advanced Java Programming

□ **UNIT 7: JAVA NETWORKING**

□ **University of Kigali (UoK)**

□ Trainer name: Dr. NTEZIRIZA NKERABAHIZI Josbert

JAVA NETWORKING



UNIT 7. JAVA NETWORKING

3

- in Java, a **socket** is a communication endpoint used to send and receive data between two computers over a network.
- More simply, a **Java socket** allows programs to talk to each other, one program acts as a **client** (requesting data) and the other as a **server** (providing data).
- Java provides socket support through the **java.net** package, mainly using two classes:
- **Socket** → used by the client to connect to a server.
- **ServerSocket** → used by the server to listen for incoming client connections.

For example, in a chat application:

- The **server** creates a **ServerSocket** and waits for connections.
- The **client** creates a **Socket** and connects to the server.
- Once connected, both can send and receive messages.
- A **Java socket** is a way for two programs to communicate over a network using client-server architecture.

UNIT 7. JAVA NETWORKING

5

- **Java Socket programming is used for communication between the applications running on different.**
- **Java Socket programming can be connection-oriented or connection-less.**
- **Socket and ServerSocket are used when there is a connection between two computers (they stay connected while communicating).**
- **DatagramSocket and DatagramPacket are used when there is no fixed connection (data is just sent without staying connected).**

UNIT 7. JAVA NETWORKING

6

- Socket / ServerSocket → with connection (like a phone call).
- DatagramSocket / DatagramPacket → no connection (like sending a message).

The client in socket programming must know two information:

- 1. IP Address of Server, and**
- 2. Port number.**

UNIT 7. JAVA NETWORKING

7

Here, we are going to make one-way client and server communication. In this application, client sends a message to the server, server reads the message and prints it. Here, two classes are being used: Socket and ServerSocket.

The Socket class is used to communicate client and server.

The ServerSocket is a class or object that is used to implement a server-side socket.

It listens for incoming client connection requests on a specified port and establishes a connection when a client connects.

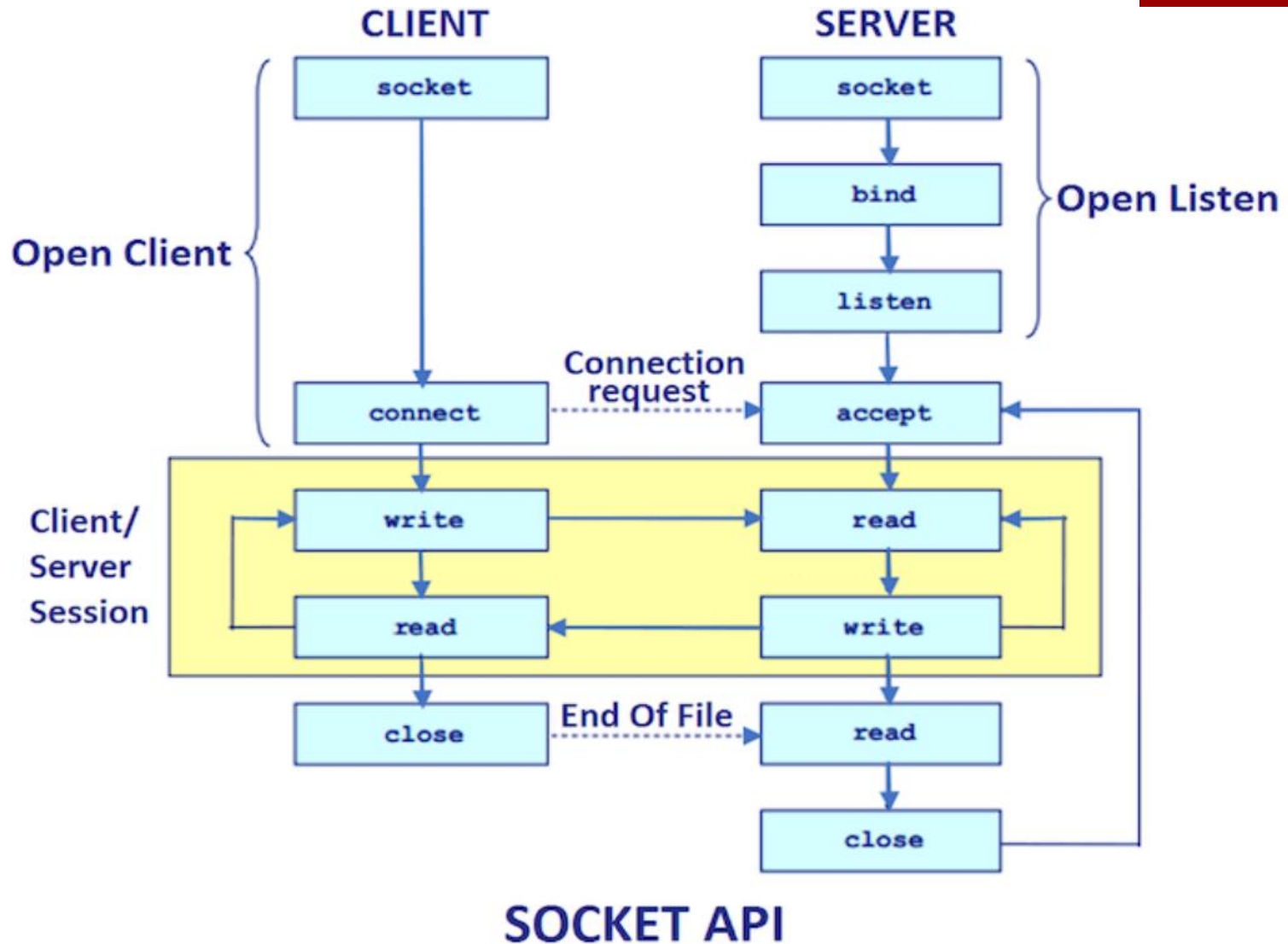
What is a socket in network programming?

A socket is like a door on a computer that lets it talk to another computer.

- Just like you use a phone to talk to someone, **a computer uses a socket to send and receive messages.**
- **Two computers can connect through sockets and talk to each other by sending data (like text, files, etc.).**

UNIT 7. JAVA NETWORKING

9



Socket class

Socket helps two computers send and receive messages over the internet or a network. The Socket class can be used to create a socket.

Important methods

Method	Description
1) public InputStream getInputStream()	returns the InputStream attached with this socket.
2) public OutputStream getOutputStream()	returns the OutputStream attached with this socket.
3) public synchronized void close()	closes this socket

DataInputStream is used for reading data from a stream, while **DataOutputStream** is used for writing data to a stream.

ServerSocket class

The ServerSocket class can be used to create a server socket. **This object is used to establish communication with the clients.**

Important methods

Method	Description
1) public Socket accept()	returns the socket and establish a connection between server and client.
2) public synchronized void close()	closes the server socket.

Creating Server:

To create the server application, we need to create the instance of **ServerSocket** class. Here, **we are using 6666 port number for the communication between the client and server.** You may also choose any other port number. **The accept() method waits for the client.** If clients connects with the given port number, it returns an instance of Socket.

```
ServerSocket ss=new ServerSocket(6666);
```

```
Socket s=ss.accept();//establishes connection and waits for the client
```

Example of Java Socket Programming

To create the client application, we need to create the instance of Socket class. Here, we need to pass the IP address or hostname of the Server and a port number. Here, we are using "localhost" because our server is running on same system.

```
Socket s=new Socket("localhost",6666);
```

Let's see a simple of Java socket programming where client sends a text and server receives and prints it.

File: MyServer.java

UNIT 7. JAVA NETWORKING

15

File: MyServer.java

Run Window Help



CheckboxExam... ListExample1... ListExample2... WindowExempl... EventApplet_... MyServer.java 99+

```
1 package josbert_new_;
2
3 import java.io.*;
4 import java.net.*;
5 public class MyServer {
6     public static void main(String[] args){
7         try{
8             ServerSocket ss=new ServerSocket(6666);
9             Socket s=ss.accept();//establishes connection
10            DataInputStream dis=new DataInputStream(s.getInputStream());
11            String str=(String)dis.readUTF();
12            System.out.println("message= "+str);
13            ss.close();
14        }catch(Exception e){System.out.println(e);}
15    }
16
17 }
```

UNIT 7. JAVA NETWORKING

16

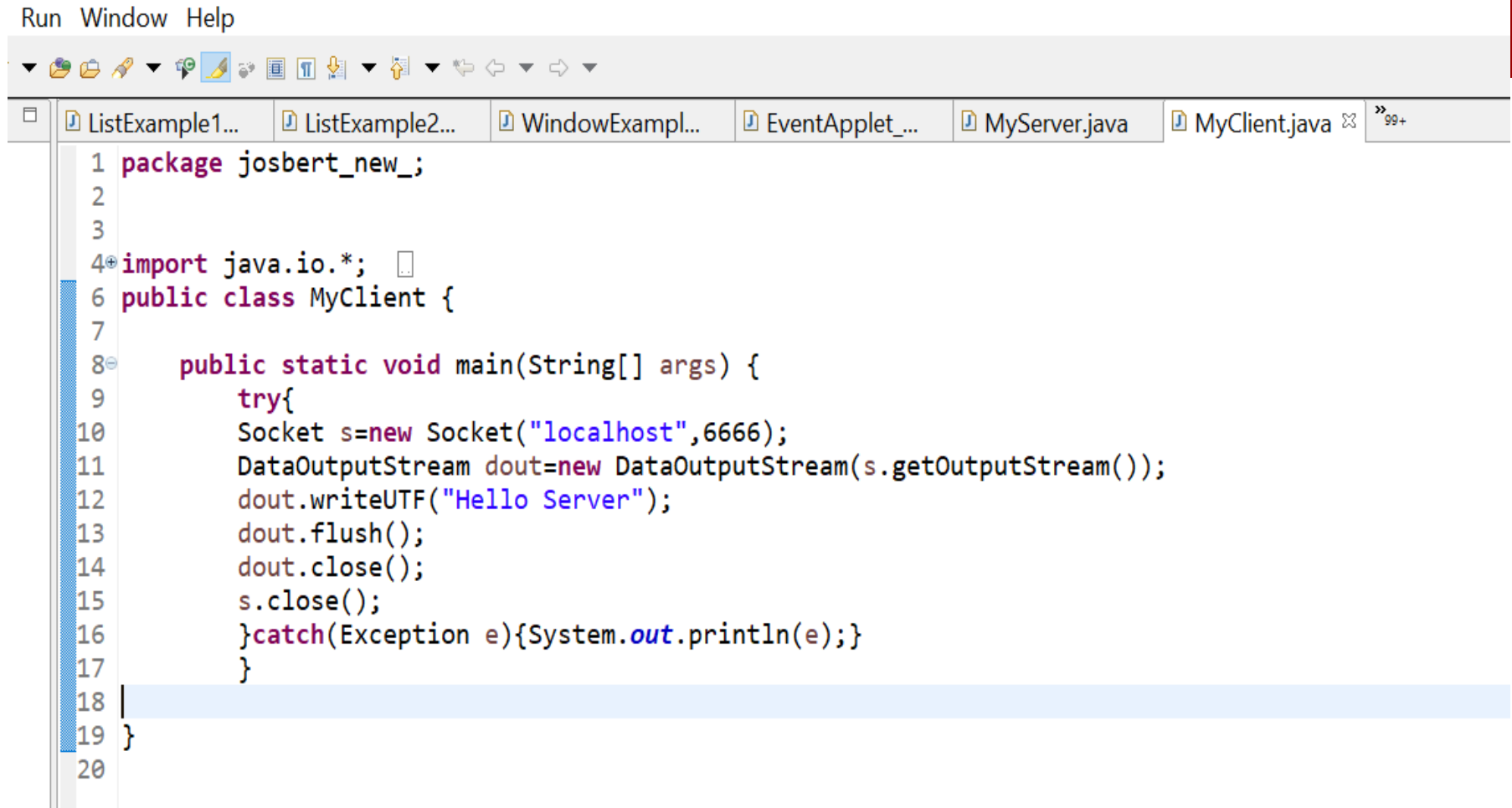
File: MyServer.java

Let's add more comments to this Java program

```
1 package josbert_new_;
2 import java.io.*; // Import necessary classes for Input/Output operations
3 import java.net.*; // Import necessary classes for networking
4 public class MyServer_cmts {
5     public static void main(String[] args) {
6         try {
7             // Create a ServerSocket that listens on port 6666
8             ServerSocket ss = new ServerSocket(6666);
9             System.out.println("Server is listening on port 6666...");
10
11             // Accept an incoming connection from a client
12             Socket s = ss.accept(); // This line blocks until a connection is established
13             System.out.println("Client connected: " + s.getInetAddress().getHostAddress());
14
15             // Create a DataInputStream to read data from the socket
16             DataInputStream dis = new DataInputStream(s.getInputStream());
17
18             // Read a UTF-encoded string sent by the client
19             String str = (String) dis.readUTF();
20             System.out.println("Message received from client: " + str);
21
22             // Close the server socket after the communication is done
23             ss.close();
24             System.out.println("Server socket closed.");
25
26         } catch (Exception e) {
27             // Print the exception if any error occurs
28             System.out.println("An error occurred: " + e);
29         }
30     }
31 }
```

Unicode Transformation Format

File: MyClient.java



```
Run Window Help
▼ ◀ ▶ ↺ ↻ 🔍 📄 📁 📂 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿
☐ ListExample1... ListExample2... WindowExempl... EventApplet_... MyServer.java MyClient.java »99+
1 package josbert_new_;
2
3
4 import java.io.*;
5
6 public class MyClient {
7
8     public static void main(String[] args) {
9         try{
10             Socket s=new Socket("localhost",6666);
11             DataOutputStream dout=new DataOutputStream(s.getOutputStream());
12             dout.writeUTF("Hello Server");
13             dout.flush();
14             dout.close();
15             s.close();
16         }catch(Exception e){System.out.println(e);}
17     }
18
19 }
20
```

UNIT 7. JAVA NETWORKING

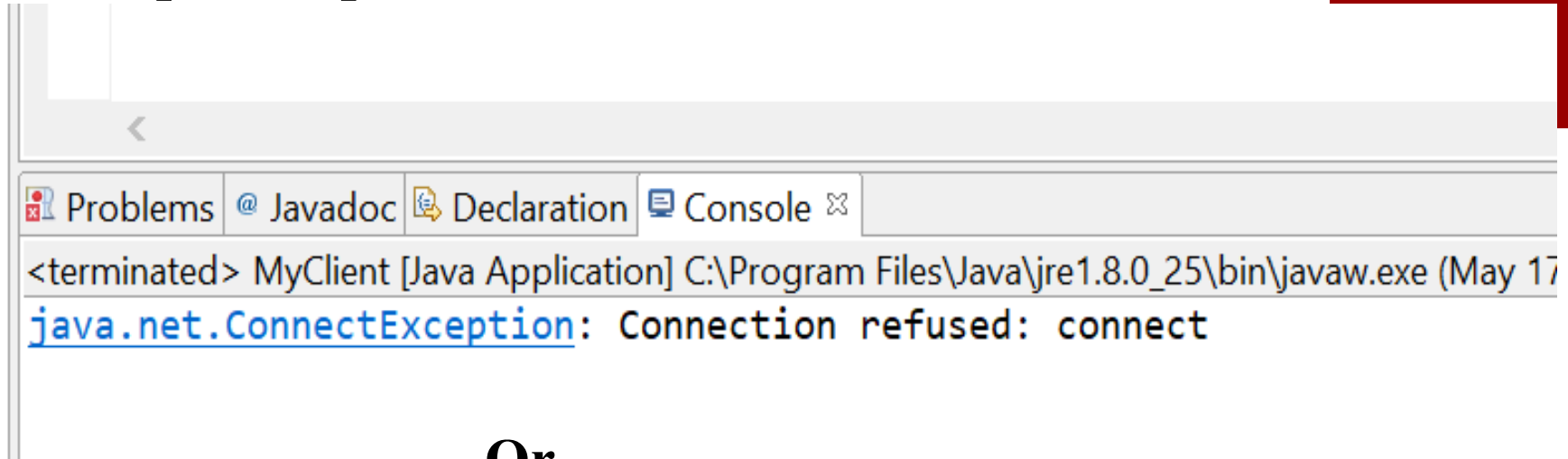
18

File: MyClient.java

Let's add more comments to this Java program

```
1 package josbert_new_;
2 import java.io.*; // Import necessary classes for Input/Output operations
3 import java.net.*; // Import necessary classes for networking operations
4 public class MyClient_ { // Define the MyClient class
5     public static void main(String[] args) { // Main method to execute the client program
6         try { // Begin try block to handle potential exceptions
7             // Create a socket connection to the server located at localhost on port 6666
8             Socket s = new Socket("localhost", 6666);
9             System.out.println("Connected to the server on port 6666."); // Notify the user
10                //that the connection was successful
11
12             // Create a DataOutputStream to send data to the server
13             // The DataOutputStream is connected to the output stream of the socket
14             DataOutputStream dout = new DataOutputStream(s.getOutputStream());
15
16             // Send a UTF-encoded string message to the server
17             dout.writeUTF("Hello Server");
18
19             // Flush the output stream to ensure all data is sent to the server
20             dout.flush();
21
22             // Close the output stream to free up resources
23             dout.close();
24
25             // Close the socket connection to the server
26             s.close();
27             System.out.println("Connection closed."); // Notify the user that the connection has been closed
28         } catch (Exception e) { // Catch any exceptions that may occur during execution
29             // Print the exception message to help with debugging
30             System.out.println("An error occurred: " + e);
31         }
32     }
```

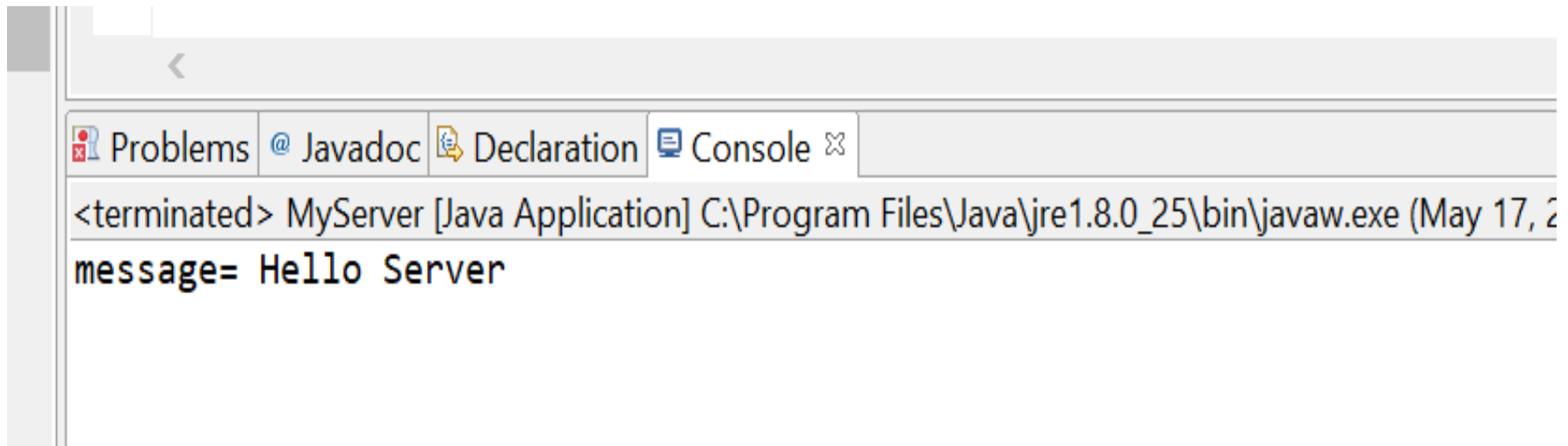
Sample Output:



A screenshot of an IDE's console window. The window has a title bar with tabs for 'Problems', '@ Javadoc', 'Declaration', and 'Console'. The 'Console' tab is active. The text in the console reads: '<terminated> MyClient [Java Application] C:\Program Files\Java\jre1.8.0_25\bin\javaw.exe (May 17, 2017 10:10:10 AM) java.net.ConnectException: Connection refused: connect'. The text is in a monospaced font, with the exception name 'java.net.ConnectException' in blue.

```
<terminated> MyClient [Java Application] C:\Program Files\Java\jre1.8.0_25\bin\javaw.exe (May 17, 2017 10:10:10 AM)
java.net.ConnectException: Connection refused: connect
```

Or



A screenshot of an IDE's console window. The window has a title bar with tabs for 'Problems', '@ Javadoc', 'Declaration', and 'Console'. The 'Console' tab is active. The text in the console reads: '<terminated> MyServer [Java Application] C:\Program Files\Java\jre1.8.0_25\bin\javaw.exe (May 17, 2017 10:10:10 AM) message= Hello Server'. The text is in a monospaced font.

```
<terminated> MyServer [Java Application] C:\Program Files\Java\jre1.8.0_25\bin\javaw.exe (May 17, 2017 10:10:10 AM)
message= Hello Server
```

Example of Java Socket Programming (Read-Write both side)

In this example, client will write first to the server then server will receive and print the text. Then server will write to the client and client will receive and print the text. The step goes on.

DataInputStream is used for reading data from a stream, while **DataOutputStream** is used for writing data to a stream.

Example of Java Socket Programming (Read-Write both side)

```
*MyServer_1....  MyClient_1.java  TestMySQL.java  MysqlCon.java  TestMySQL_.java  TestM
1 package josbert_new_;
2 import java.net.*;
4 public class MyServer_1 {
5
6     public static void main(String args[])throws Exception{
7         ServerSocket ss=new ServerSocket(3333);
8         Socket s=ss.accept();
9         DataInputStream din=new DataInputStream(s.getInputStream());
10        DataOutputStream dout=new DataOutputStream(s.getOutputStream());
11        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
12
13        String str="",str2="";
14        while(!str.equals("stop")){ // while not equal stop
15            str=din.readUTF();
16            System.out.println("client says: "+str);
17            str2=br.readLine();
18            dout.writeUTF(str2);
19            dout.flush();
20        }
21        din.close();
22        s.close();
23        ss.close();
24    }
25 }
```

Server side

UNIT 7. JAVA NETWORKING

22

Let's add more comments to this Java program

```
1 package josbert_new_;
2 import java.net.*; // Import necessary classes for networking operations
3 import java.io.*; // Import necessary classes for Input/Output operations
4 public class MyServer_1_cmts {
5     // Define the MyServer_1 class
6     public static void main(String args[]) throws Exception { // Main method to run the server
7         // Create a ServerSocket that listens on port 3333 for incoming client connections
8         ServerSocket ss = new ServerSocket(3333);
9         System.out.println("Server started, waiting for clients to connect...");
10
11         // Wait for a client to connect and accept the connection, creating a Socket object
12         Socket s = ss.accept();
13         System.out.println("Client connected: " + s.getRemoteSocketAddress()); // Print the client
14         // socket address for reference
15
16         // Create a DataInputStream to read data sent from the client
17         DataInputStream din = new DataInputStream(s.getInputStream());
18
19         // Create a DataOutputStream to send data to the client
20         DataOutputStream dout = new DataOutputStream(s.getOutputStream());
21
22         // Create a BufferedReader to read input from the server's console
23         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
24
25         // Initialize variables to hold the received message and the response message
26         String str = "", str2 = "";
27
28         // Continuously listen for messages from the client until the server receives "stop"
29         while (!str.equals("stop")) { // Loop will continue until "stop" is received
30             // Read a UTF-encoded string message from the client
31             str = din.readUTF();
```

Server side

```
32
33 // Print the message received from the client to the server console
34 System.out.println("Client says: " + str);
35
36 // Read a response from the server's console using BufferedReader
37 str2 = br.readLine();
38
39 // Send the response message back to the client
40 dout.writeUTF(str2);
41 dout.flush(); // Flush the output stream to ensure all data is sent immediately
42 }
43
44 // Close the input stream, socket, and server socket to release resources
45 din.close(); // Close the DataInputStream
46 s.close();   // Close the client Socket
47 ss.close();  // Close the ServerSocket
48 System.out.println("Server closed."); // Print message indicating the server has shut down
49 }
50 }
```

Server side

UNIT 7. JAVA NETWORKING

24

Example of Java Socket Programming (Read-Write both side)

Run Window Help



WindowExempl... EventApplet... MyServer.java MyClient.java MyServer_1.java MyClient_1.java

```
3 import java.net.*;
4 import java.io.*;
5
6 public class MyClient_1 {
7
8     public static void main(String args[]) throws Exception {
9         Socket s=new Socket("localhost",3333);
10        DataInputStream din=new DataInputStream(s.getInputStream());
11        DataOutputStream dout=new DataOutputStream(s.getOutputStream());
12        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
13
14        String str="",str2="";
15        while(!str.equals("stop")){
16            str=br.readLine();
17            dout.writeUTF(str);
18            dout.flush();
19            str2=din.readUTF();
20            System.out.println("Server says: "+str2);
21        }
22
23        dout.close();
24        s.close();
25    }
26
27 }
```

Client side


```
1 package josbert_new_;
2 import java.net.*; // Import necessary classes for networking operations
3 import java.io.*; // Import necessary classes for Input/Output operations
4 public class MyClient_1_cmts { // Define the MyClient_1 class
5     public static void main(String args[]) throws Exception { // Main method to run the client
6         // Create a Socket object to connect to the server at "localhost" on port 3333
7         Socket s = new Socket("localhost", 3333);
8         System.out.println("Connected to server at localhost:3333"); // Notify the user of successful connection
9
10        // Create a DataInputStream to read messages sent from the server
11        DataInputStream din = new DataInputStream(s.getInputStream());
12
13        // Create a DataOutputStream to send messages to the server
14        DataOutputStream dout = new DataOutputStream(s.getOutputStream());
15
16        // Create a BufferedReader to read user input from the console
17        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
18
19        // Initialize variables to hold the received message and the response message
20        String str = "", str2 = "";
21
22        // Continuously read input from the user until "stop" is entered
23        while (!str.equals("stop")) { // Loop continues until the user types "stop"
24            // Read a line of input from the console (user input)
25            str = br.readLine();
26
27            // Send the user's input to the server as a UTF-encoded string
28            dout.writeUTF(str);
29            dout.flush(); // Flush the output stream to ensure the message is sent immediately
30
31            // Read the response from the server
32            str2 = din.readUTF();
```

Client side

UNIT 7. JAVA NETWORKING

26

Let's add more comments to this Java program

```
33
34     // Print the server's response to the console
35     System.out.println("Server says: " + str2);
36 }
37
38 // Close the output stream and socket connection to release resources
39 dout.close(); // Close the DataOutputStream
40 s.close();    // Close the socket connection with the server
41 System.out.println("Connection closed."); // Notify the user that the connection has been closed
42 } |
43
44 }
```

Client side

In the server-client model, multiple clients connect to a server, and they communicate with the server to exchange information.

The server runs continuously, listening for clients to make connection requests. The basic steps in setting up a server and client are:

Server:

Instantiate a `ServerSocket`, assigning it a specific port to monitor for incoming requests. Use the **`ServerSocket's accept() method`** to await a client's connection, which stops execution until a connection is established.

After accepting a connection, the server can interact with the client using the `Socket` object that the **`accept() method`** provides.

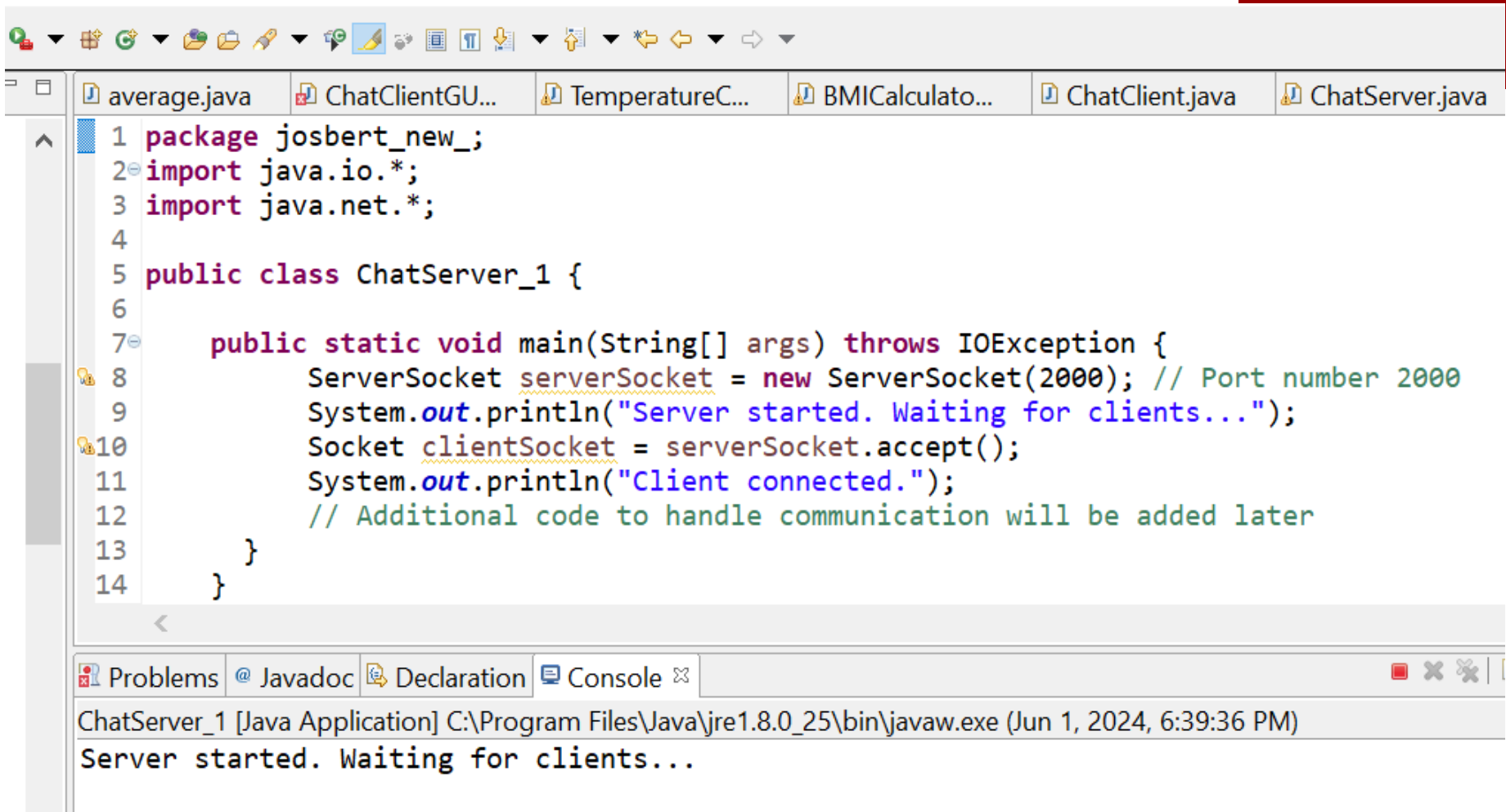
Client:

Initialize a Socket object with the server's hostname and port number to establish a connection.

After the connection is successfully established, the client can use the Socket object to communicate with the server.

Next, let's examine some code examples to illustrate these concepts in action.

Server:



The screenshot shows an IDE with a toolbar at the top. Below the toolbar, a tab bar contains several files: `average.java`, `ChatClientGU...`, `TemperatureC...`, `BMICalculato...`, `ChatClient.java`, and `ChatServer.java`. The `ChatServer.java` file is active, displaying the following code:

```
1 package josbert_new_;
2 import java.io.*;
3 import java.net.*;
4
5 public class ChatServer_1 {
6
7     public static void main(String[] args) throws IOException {
8         ServerSocket serverSocket = new ServerSocket(2000); // Port number 2000
9         System.out.println("Server started. Waiting for clients...");
10        Socket clientSocket = serverSocket.accept();
11        System.out.println("Client connected.");
12        // Additional code to handle communication will be added later
13    }
14 }
```

At the bottom of the IDE, there is a panel with tabs for `Problems`, `Javadoc`, `Declaration`, and `Console`. The `Console` tab is selected, showing the output of the program:

```
ChatServer_1 [Java Application] C:\Program Files\Java\jre1.8.0_25\bin\javaw.exe (Jun 1, 2024, 6:39:36 PM)
Server started. Waiting for clients...
```

Client:



```
average.java  TemperatureC...  BMICalculato...  ChatClient.java  ChatServer.java  ChatServer_1...  ChatClient.java
1 package josbert_new_;
2
3 import java.io.*;
4 import java.net.*;
5
6 public class ChatClient_1 {
7
8     public static void main(String[] args) throws IOException {
9         Socket socket = new Socket("localhost", 5000); // Connect to server on port 5000
10        System.out.println("Connected to server.");
11        // Additional code to handle communication will be added later
12    }
13
14 }
15
```

Creating the Chat Server

Now, let's get to work and start creating the chat server, which will act as the central hub for all client communications. This will accept connections from clients, receive messages from one client, and broadcast those messages to all other connected clients.

Spawning a New Thread for Each Client

To allow multiple clients to connect and communicate simultaneously, the server will generate a new thread for each client connection.

This thread will handle all communication with that client, allowing the server to remain responsive and accept additional connections.

UNIT 7. JAVA NETWORKING

32

Creating the Chat Server

Now we've outlined what we need to do, here is the basic implementation of our chat server in Java:

```
1 package josbert_new_;
2 import java.io.*;
3 import java.net.*;
4 import java.util.*;
5
6 public class ChatServer_2 {
7
8     // List to keep track of all connected clients
9     private static List<ClientHandler> clients = new ArrayList<>();
10
11     public static void main(String[] args) throws IOException {
12         ServerSocket serverSocket = new ServerSocket(5000);
13         System.out.println("Server started. Waiting for clients...");
14
15         while (true) {
16             Socket clientSocket = serverSocket.accept();
17             System.out.println("Client connected: " + clientSocket);
18
19             // Spawn a new thread for each client
20             ClientHandler clientThread = new ClientHandler(clientSocket, clients);
21             clients.add(clientThread);
22             new Thread(clientThread).start();
23         }
24     }
25
26     class ClientHandler implements Runnable {
27         private Socket clientSocket;
28         private List<ClientHandler> clients;
29         private PrintWriter out;
30     }
```



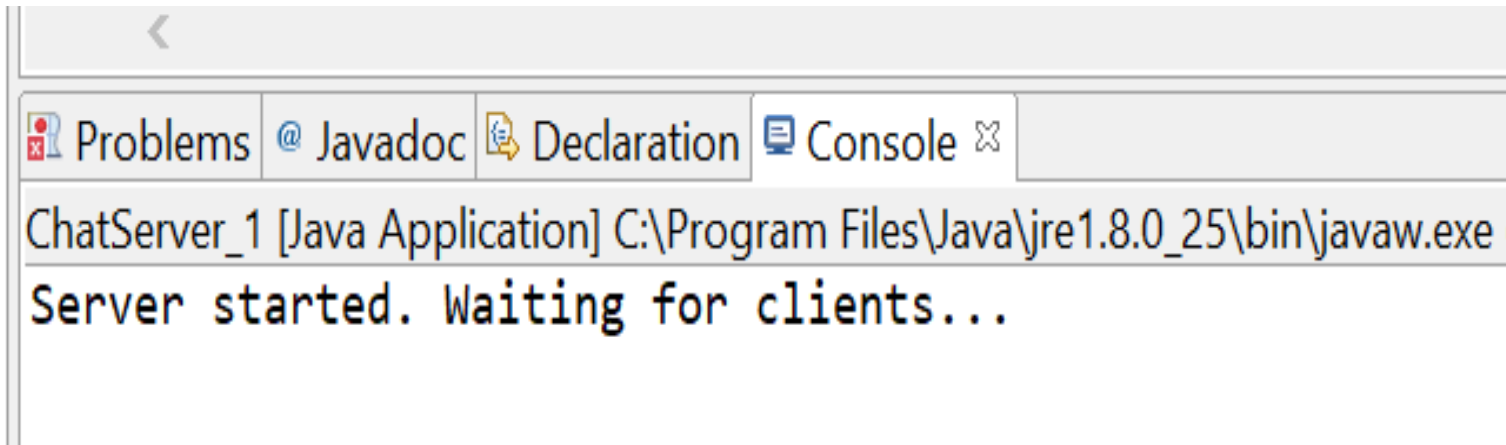
```
31     private BufferedReader in;
32
33     public ClientHandler(Socket socket, List<ClientHandler> clients) throws IOException {
34         this.clientSocket = socket;
35         this.clients = clients;
36         this.out = new PrintWriter(clientSocket.getOutputStream(), true);
37         this.in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
38     }
39
40     public void run() {
41         try {
42             String inputLine;
43             while ((inputLine = in.readLine()) != null) {
44                 // Broadcast message to all clients
45                 for (ClientHandler aClient : clients) {
46                     aClient.out.println(inputLine);
47                 }
48             }
49         } catch (IOException e) {
50             System.out.println("An error occurred: " + e.getMessage());
51         } finally {
52             try {
53                 in.close();
54                 out.close();
55                 clientSocket.close();
56             } catch (IOException e) {
```

UNIT 7. JAVA NETWORKING

34

```
57         e.printStackTrace();
58     }
59 }
60 }
61 }
```

Sample Output:



The screenshot shows an IDE interface with a console window. The console has tabs for 'Problems', 'Javadoc', 'Declaration', and 'Console'. The 'Console' tab is active, displaying the output of a Java application. The output text is 'ChatServer_1 [Java Application] C:\Program Files\Java\jre1.8.0_25\bin\javaw.exe' followed by 'Server started. Waiting for clients...'.

```
ChatServer_1 [Java Application] C:\Program Files\Java\jre1.8.0_25\bin\javaw.exe
Server started. Waiting for clients...
```

Let's add more comments to this Java program

```
1 package josbert_new_;
2 import java.io.*;           // Import necessary classes for Input/Output operations
3 import java.net.*;          // Import necessary classes for networking operations
4 import java.util.*;         // Import utility classes, such as List and ArrayList
5 public class Chat_server_cmts {
6     // List to keep track of all connected clients
7     private static List<ClientHandler> clients = new ArrayList<>(); // Initializes an empty
8     //list for client handlers
9
10    public static void main(String[] args) throws IOException {
11        // Create a ServerSocket that listens on port 5000 for incoming client connections
12        ServerSocket serverSocket = new ServerSocket(5000);
13        System.out.println("Server started. Waiting for clients..."); // Notify that the server is ready
14
15        // Main server loop, waiting for client connections
16        while (true) {
17            // Accept a new client connection and create a corresponding Socket
18            Socket clientSocket = serverSocket.accept();
19            System.out.println("Client connected: " + clientSocket); // Log client connection
20
21            // Spawn a new thread for each client using a ClientHandler
22            ClientHandler clientThread = new ClientHandler(clientSocket, clients);
23            clients.add(clientThread); // Add the new client handler to the list of clients
24            new Thread(clientThread).start(); // Start the client handler thread to manage
25            //client communication
26        }
27    }
28 }
29
30 // Class that handles communication with a connected client
31 class ClientHandler implements Runnable {
32     private Socket clientSocket; // Socket for the client
```

```
33 private List<ClientHandler> clients;           // List to keep track of all client handlers
34 private PrintWriter out;                       // Output stream for sending messages to the client
35 private BufferedReader in;                     // Input stream for receiving messages from the client
36
37 // Constructor to initialize the ClientHandler with the client socket and list of clients
38 public ClientHandler(Socket socket, List<ClientHandler> clients) throws IOException {
39     this.clientSocket = socket;                // Assign the client socket to the instance variable
40     this.clients = clients;                    // Assign the list of clients
41     this.out = new PrintWriter(clientSocket.getOutputStream(), true); // Initialize the output stream
42     this.in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream())); // Initialize
43     // the input stream
44 }
45
46 // Run method that is executed when the thread starts
47 public void run() {
48     try {
49         String inputLine; // Variable to hold messages from the client
50         // Continuously read messages from the client
51         while ((inputLine = in.readLine()) != null) {
52             // Broadcast message to all clients in the list
53             for (ClientHandler aClient : clients) {
54                 aClient.out.println(inputLine); // Send the received message to each client
55             }
56         }
57     }
58 }
```

```
57 } catch (IOException e) {
58     // Handle any I/O exceptions that occur during communication
59     System.out.println("An error occurred: " + e.getMessage());
60 } finally {
61     // Clean up resources when the client disconnects
62     try {
63         in.close();           // Close the input stream
64         out.close();          // Close the output stream
65         clientSocket.close(); // Close the client socket connection
66     } catch (IOException e) {
67         e.printStackTrace(); // Print the stack trace for any exceptions thrown during cleanup
68     }
69 }
70 }
71 }
```

Creating the Chat Client

Now we have our chat server up and running, it's time to create the client part of our application.

The client will connect to the server, send messages, and receive broadcasts from other clients. Let's dive into the process.

The client should be able to send messages to the server. This involves setting up an output stream and writing messages to it.

The client also needs to continuously listen for messages from the server. This can be done by setting up an input stream and reading from it in a loop.

Now we've outlined what we need to do, here is the basic implementation of our chat client in Java:

UNIT 7. JAVA NETWORKING

39

Creating the Chat Client

Example

```
ChatClientGU... TemperatureC... BMICalculato... ChatClient.java ChatServer.java ChatServer_1... »»+
1 package josbert_new_;
2
3 import java.io.*;
4 import java.net.*;
5 public class ChatClient {
6
7     private Socket socket = null;
8     private BufferedReader inputConsole = null;
9     private PrintWriter out = null;
10    private BufferedReader in = null;
11
12    public ChatClient(String address, int port) {
13        try {
14            socket = new Socket(address, port);
15            System.out.println("Connected to the chat server");
16
17            inputConsole = new BufferedReader(new InputStreamReader(System.in));
18            out = new PrintWriter(socket.getOutputStream(), true);
19            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
20
21            String line = "";
22            while (!line.equals("exit")) {
23                line = inputConsole.readLine();
24                out.println(line);
25                System.out.println(in.readLine());
26            }
27        }
28    }
29 }
```

UNIT 7. JAVA NETWORKING

40

Creating the Chat Client

```
27
28     socket.close();
29     inputConsole.close();
30     out.close();
31 } catch (UnknownHostException u) {
32     System.out.println("Host unknown: " + u.getMessage());
33 } catch (IOException i) {
34     System.out.println("Unexpected exception: " + i.getMessage());
35 }
36 }
37
38 }
39
```

Problems @ Javadoc Declaration Console

ChatClient [Java Application] C:\Program Files\Java\jre1.8.0_25\bin\javaw.exe (Jun 1, 2024, 7:34:39 PM)

Connected to the chat server

UNIT 7. JAVA NETWORKING

41

Let's add more comments to this Java program

```
1 package josbert_new_;
2 import java.io.*;          // Import classes for Input/Output operations
3 import java.net.*;         // Import classes for networking operations
4 public class ChatClient_cmts {
5     private Socket socket = null;          // Socket for connecting to the server
6     private BufferedReader inputConsole = null; // Reader for input from the console (user input)
7     private PrintWriter out = null;        // Writer for sending messages to the server
8     private BufferedReader in = null;      // Reader for receiving messages from the server
9
10    // Constructor for ChatClient that takes server address and port number as parameters
11    public ChatClient_cmts(String address, int port) {
12        try {
13            // Create a socket connecting to the specified address and port
14            socket = new Socket(address, port);
15            System.out.println("Connected to the chat server"); // Notify the user of successful connection
16
17            // Initialize input reader for reading user input from the console
18            inputConsole = new BufferedReader(new InputStreamReader(System.in));
19            // Initialize PrintWriter for sending output to the server
20            out = new PrintWriter(socket.getOutputStream(), true);
21            // Initialize BufferedReader for reading input from the server
22            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
23
24            // Variable to store user input
25            String line = "";
26            // Continuously read user input until "exit" is entered
27            while (!line.equals("exit")) {
28                // Read a line of input from the console
29                line = inputConsole.readLine();
30                // Send the input line to the server
31                out.println(line);
```

```
32         // Read and print the server's response
33         System.out.println(in.readLine());
34     }
35
36     // Close all opened resources before exiting
37     socket.close();           // Close the socket connection to the server
38     inputConsole.close();     // Close the console input stream
39     out.close();              // Close the PrintWriter stream
40 } catch (UnknownHostException u) {
41     System.out.println("Host unknown: " + u.getMessage()); // Handle case where the
42     //host cannot be determined
43 } catch (IOException i) {
44     System.out.println("Unexpected exception: " + i.getMessage()); // Handle I/O
45     //exceptions during setup or communication
46 }
47 }
48 }
```

